



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 9
по курсу «Языки и методы программирования»
«DSU (Disjoint Set Union) в C++»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Класс Element	1
dsu.hpp	2
Операторы	2
Operator dereference	2
Оператор find !	3
Метод find()	3
Оператор сравнения ==	3
Объединение множеств	4
Оператор <<	4
main()	5
Вывод	6

Цель работы

Реализовать структуру данных DSU (Disjoint Set Union, система непересекающихся множеств) на C++. DSU используется для эффективного объединения множеств и проверки принадлежности одному множеству.

Задачи:

- Изучить структуру данных DSU.
- Реализовать класс Element с find и union.
- Использовать перегрузку операторов.
- Применить path compression и union by rank.

Теоретические сведения

DSU (Disjoint Set Union) — структура данных для работы с непересекающимися множествами. Основные операции:

- make_set(x) — создание нового множества
- find(x) — поиск представителя множества
- union(x, y) — объединение множеств

Оптимизации:

- Path compression — сжатие пути при find
- Union by rank — объединение по рангу

Класс Element

Шаблонный класс Element представляет элемент множества.

dsu.hpp

dsu.hpp

```
template <typename T> class Element {
private:
    T value;
    Element<T> *parent;
    int rank;

public:
    Element(T val) : value(val), parent(this), rank(0) {}

    T &operator*() { return value; }

    Element<T> *operator!() { ... }

    Element<T> *find() { ... }

    Element<T> *find() const { ... }

    bool operator==(const Element<T> &other) const { ... }

    bool operator!=(const Element<T> &other) const { ... }

    void operator<<(Element<T> &other) { ... }
};
```

- value — хранимое значение
- parent — указатель на родителя (в дереве)
- rank — ранг для оптимизации union

Операторы

Operator dereference

dsu.hpp

```
T &operator*() { return value; }
```

Возвращает ссылку на хранимое значение.

Оператор find !

dsu.hpp

```
Element<T> *operator!() {  
    if (parent == this) {  
        return this;  
    }  
    return parent = !(*parent);  
}
```

Перегрузка оператора ! для find с path compression. При вызове возвращает корень множества и сжимает путь.

Метод find()

dsu.hpp

```
Element<T> *find() {  
    if (parent == this) {  
        return this;  
    }  
    return parent = parent->find();  
}  
  
Element<T> *find() const {  
    if (parent == this) {  
        return const_cast<Element<T> *>(this);  
    }  
    return parent->find();  
}
```

Два варианта: изменяемый и константный. Рекурсивный поиск корня с path compression.

Оператор сравнения ==

dsu.hpp

```
bool operator==(const Element<T> &other) const {  
    return find() == const_cast<Element<T> &>(other).find();  
}  
  
bool operator!=(const Element<T> &other) const {  
    return !(*this == other);  
}
```

Проверка принадлежности одному множеству через сравнение корней.

Объединение множеств

Оператор <<

dsu.hpp

```
void operator<<(Element<T> &other) {
    Element<T> *root1 = find();
    Element<T> *root2 = other.find();

    if (root1 != root2) {
        if (root1->rank < root2->rank) {
            root1->parent = root2;
        } else if (root1->rank > root2->rank) {
            root2->parent = root1;
        } else {
            root2->parent = root1;
            root1->rank++;
        }
    }
}
```

Перегрузка оператора << для объединения множеств (аналог union).

Алгоритм union by rank:

- Если $\text{rank}_{\text{root1}} < \text{rank}_{\text{root2}}$ — root1 становится child root2
- Если $\text{rank}_{\text{root1}} > \text{rank}_{\text{root2}}$ — root2 становится child root1
- Если равны — выбираем любой, увеличиваем его rank

main()

main.cpp

```
int main() {
    Element<int> a(1);
    Element<int> b(2);
    Element<int> c(3);
    Element<int> d(4);

    auto print = [&]() {
        std::cout << "  a = " << *a << ", root: " << **!a << std::endl;
        std::cout << "  b = " << *b << ", root: " << **!b << std::endl;
        std::cout << "  c = " << *c << ", root: " << **!c << std::endl;
        std::cout << "  d = " << *d << ", root: " << **!d << std::endl;
        std::cout << "——\n";
    };

    print();
    a << b;
    std::cout << "a << b\n";
    print();
    c << d;
    std::cout << "c << d\n";
    print();
    a << c;
    std::cout << "a << c\n";
    print();
}
```

Вывод:

```
a = 1, root: 1
b = 2, root: 2
c = 3, root: 3
d = 4, root: 4
——
a << b
a = 1, root: 1
b = 2, root: 1
c = 3, root: 3
d = 4, root: 4
——
c << d
a = 1, root: 1
b = 2, root: 1
c = 3, root: 3
d = 4, root: 3
——
```

```
a << c
a = 1, root: 1
b = 2, root: 1
c = 3, root: 1
d = 4, root: 1
—
```

Вывод

В ходе лабораторной работы была реализована структура данных DSU:

1. Класс Element — шаблонный класс для элементов множества
2. Path compression — оптимизация поиска корня
3. Union by rank — оптимизация объединения
4. Operator overloading - use operator names instead

DSU применяется в алгоритмах Крускала, connected components, и других задачах с объединением множеств.